Week 10 homework: Campus service request system

1. Understand the problem

1.1. User roles

- Student: The person who creates and tracks support requests.
- Staff: The staff member who receives, processes, and updates the status of requests.

1.2. Requirements list

- Account management:
- The system allows Students and Staff to log in using the school's internal accounts.
- The system shows different interfaces based on the user's role (Student or Staff).
- Submit a request:
- Students can create a new "Service Request"
- When creating a request, Students must fill in: Title, Description, and Location (e.g., Room P401).
- The system automatically generates a unique identifier (Request ID) for each newly created request.
- The system automatically sets the default status of a new request to "Pending."
- The system also automatically records the created date/time (Created At) and the creator (Created By).
- View a list of requests:
- Students can view a list of all requests they have created.
- Staff can view all requests from all students.
- Staff can filter the request list by status (Pending, In Progress, Done).
- View request details:
- Users can click on a specific request to see full details.
- The system automatically disables editing of the request content by Students once the status moves beyond "Pending" in order to protect the integrity of the original issue data.
- Update request status:
- Staff can change the status of a request.
- The valid status flow is: Pending → In Progress → Done.

- The system records the last updated time (Updated At).

2. Use cases

- Use case 1: Create a new service request
- Use case name: Submit Request
- Actor: Student
- Trigger: The student notices an issue (e.g., a broken projector) and accesses the system to report it.
- Main flow:
    - The student logs in successfully and selects the “Create New Request” function.
    - The system displays an input form.
    - The student fills in all required fields: Title, Description, and Location.
    - The student clicks the “Submit” button.
    - The system validates the input data.
    - The system creates a new record in the database, generates a unique ID, sets the initial status to “Pending,” and records the created time (Created At).
- Alternative flow: If the student leaves any required field empty, the system refuses to save the data and shows a message asking the user to complete all fields.
- Success outcome: A valid new request is created and stored in the system.
- Use case 2: View and filter request list
- Use case name: View Request
- Actor: Staff
- Trigger: The staff member starts their shift and needs to check pending requests.
- Main flow:
    - The staff logs in successfully and accesses the “Staff Dashboard.”
    - The system retrieves all request data from all students.
    - The system returns a list sorted by the most recent time.
    - The staff uses the filter and selects the status “Pending.”
    - The system processes the query and displays only the matching records.
- Success outcome: The staff can view the correct list of requests based on the selected criteria.
- Use case 3: View request details

- Use case name: View Request Details
- Actor: Student/Staff
- Trigger: The user wants to see full information and history of a specific request.
- Main flow:
    + From the list screen, the user clicks on a Request ID.
    + The system receives the Request ID and queries detailed data from the database.
    + The system returns all original information (Title, Description, Location) along with system history (current status, created by, created date, updated date).
- Alternative flow: If the Request ID does not exist or has been deleted due to a system error, the system returns a “404 Not Found” error message.
- Success outcome: All detailed and transparent information about the issue is displayed to the user.

• Use case 4: Update request status

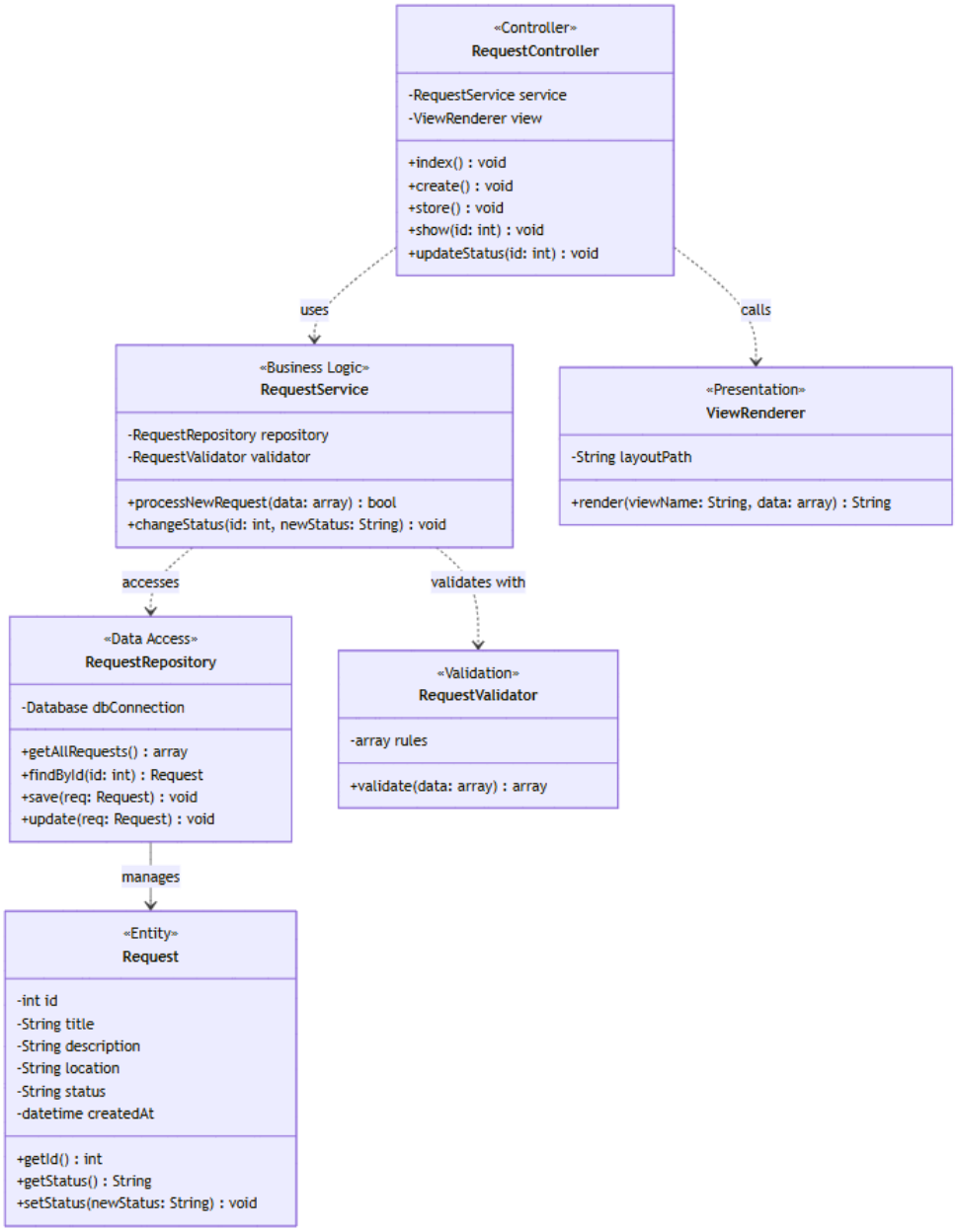
- Use case name: Update Request Status
- Actor: Staff
- Trigger: The staff member starts fixing the issue or has completed it and needs to update the system.
- Main flow:
    + The staff opens the detail view of a request (currently in “Pending” status).
    + The staff changes the status to “In Progress” or “Done.”
    + The staff enters information in the “Resolution Note” field.
    + The staff clicks the “Update” button.
    + The system checks the validity of the status flow and validates the Resolution Note field.
    + The system saves the new status, records the staff ID, and updates the timestamp (Updated At).
- Alternative flow: If the staff selects “Done” but leaves the “Resolution Note” empty, the system blocks the action and requires the user to enter a resolution before closing the request.

– Success outcome: The request status is successfully updated according to the process.

3. SRP class responsibility table

Class name	Responsibility	Reason to change
Request *(Entity)*	Stores the data structure of a service request (Title, Description, Status, CreatedAt, etc.).	When the school wants to add or remove data fields (e.g., adding an "Attachment Image" field).
RequestRepository *(Data Access)*	Executes database queries (SQL) such as insert, retrieve list, and update.	When the system changes the database type (e.g., from MySQL to PostgreSQL) or needs query optimization.
RequestValidator *(Validation)*	Validates input data (e.g., Title must not be empty, maximum length is 255 characters).	When validation rules are updated or changed.
RequestService *(Business Logic)*	Handles core business rules (e.g., checking valid status flow, requiring a Resolution Note when status is "Done").	When the issue-handling process changes (e.g., allowing skipping status steps).
RequestController *(Coordinator)*	Receives user requests (URL/HTTP), calls other classes to process logic, and selects the appropriate View to display.	When the web navigation flow changes (e.g., redirecting to a different page after successful submission).
ViewRenderer *(Presentation)*	Reads template files (HTML/PHP) and injects data to generate the final web page.	When the rendering mechanism or layout structure of the system changes.

4. UML diagram



5. MVC mapping – Route table

5.1. MVC mapping table

Item	MVC Layer	Why?
Request	Model	Represents the core data structure of the application.
RequestRepository	Model	Directly interacts with the database to retrieve and store data.

RequestValidator & RequestService	Model	Contains all business logic and data validation rules (in pure MVC, this logic is grouped in the Model layer).
RequestController	Controller	Acts as a coordinator: receives requests, calls the Model to get data, and passes the data to the View.
requests/index.php	View	Contains HTML code to display the request list interface.
requests/show.php	View	Contains HTML code to display the detailed view of a request.

5.2. Route table

HTTP Method	URL	Controller @ Action	Purpose
GET	/requests/create	RequestController@create	(Student) Displays the form to create a new service request.
POST	/requests/create	RequestController@store	(Student) Handles saving form data to the database.
GET	/requests	RequestController@index	(Student) Displays the list of requests created by the student.
GET	/staff/requests	RequestController@staffIndex	(Staff) Displays the full list of all requests in the system for management.
GET	/requests/{id}	RequestController@show	(All) Displays detailed information of a specific request by ID.
POST	/requests/{id}/status	RequestController@updateStatus	(Staff) Updates the request status (Pending → In Progress → Done).

6. Minimal skeleton

6.1. public/index.php

```
echo "Hello MVC - Campus Service Request System";
```

6.2. app/Models/Request.php

```
Class Request:

  // properties: id, title, description, status (camelCase)
```

6.3. app/Models/RequestService.php

```
Class RequestService:
  Method changeStatus(id, newStatus):
    - validate status transition rules
    - repository.update(id, newStatus)
```

6.4. app/Models/RequestController.php

```
Class RequestController:
  RequestController@staffIndex():
    - fetch all requests from database
    - load view requests/index.php

  RequestController@updateStatus(id):
    - read newStatus from POST
    - service.changeStatus(id, newStatus)
    - redirect to /requests/{id}
```

6.5. app/Views/requests/index.php

```
<h1>Campus Service Requests</h1>
```

6.6. app/Views/requests/show.php

```
<h1>Request Details</h1>
```

7. Reflection

7.1. Which parts of your design are Model, View, and Controller?

- Model: This layer includes Request (data entity), RequestRepository (data access/database interactions), and RequestService (business logic). It represents the core data and rules of the application.
- View: This layer consists of the HTML/PHP templates located in the app/Views/requests/ folder (such as index.php and show.php). Its sole responsibility is to render the user interface.

- Controller: The RequestController acts as the coordinator. It intercepts incoming HTTP requests, invokes the Model to process data, and passes the result to the appropriate View for display.

7.2. Where should validation happen, and why?

Validation should happen within the Model layer (specifically inside a RequestValidator or RequestService class) before the data reaches the repository. This ensures that business rules and data integrity are enforced consistently across the system, regardless of where the input comes from. It also keeps the Controller thin, strictly adhering to the Single Responsibility Principle (SRP) by preventing the Controller from handling business logic.

7.3. What would break if you put SQL inside a View file?

Putting SQL inside a View file fundamentally breaks the MVC architecture and the Single Responsibility Principle. It creates "spaghetti code" by tightly coupling the presentation UI with the data access layer. Consequently, any future database changes would require modifying UI files, making the system extremely difficult to maintain, debug, and scale. Furthermore, it exposes the application to severe security risks, such as SQL Injection vulnerabilities.

7.4. What code do you expect to write next week to make this real?

To transition this design into a working application next week, I expect to write:

- A Router mechanism in public/index.php to map incoming URLs to specific Controller actions.
- An Autoloader script to automatically load required class files without manually writing require_once.
- The actual PHP implementation inside the RequestController methods to handle HTTP GET/POST requests.
- The concrete SQL queries inside the RequestRepository (using PDO) to execute CRUD operations on the database.